

P2025R0

Guaranteed copy elision for named return objects

Published Proposal, 2020-01-08

Issue Tracking:

[Inline In Spec](#)

Author:

[Anton Zhilin](#)

Audience:

SG17, EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

<https://github.com/Anton3/cpp-proposals/blob/master/published/p2025r0.bs>

Current:

<https://github.com/Anton3/cpp-proposals/blob/master/published/p2025r0.html>

Abstract

This proposal aims to provide guaranteed copy elision for common cases of local variables being returned from a function.

Table of Contents

1 Motivation

2 Proposed solution

2.1 Examples

2.2 Proposed wording

3 Frequently Asked Questions

3.1 Are the proposed changes source or ABI breaking?

3.2 What are the costs associated with the proposed changes?

3.3 What about trivially-copyable temporaries?

3.4 Is "named return object" a good term choice?

4 Alternative solutions

4.1 Implement similar functionality using existing features

4.2 Guarantee NRVO in more cases

4.3 Require an explicit mark for named return objects

4.4 Alias expressions

5 Future work

5.1 Guarantee some other types of copy elision

5.2 Guarantee currently disallowed types of copy elision

5.3 Reduce the number of moves performed in other cases

6 Acknowledgements

Index

Terms defined by this specification

References

Normative References

Informative References

Issues Index

§ 1. Motivation

The accepted [\[P0135R0\]](#) proposal already provides guaranteed copy elision for when a prvalue is returned from a function by stating that the result object of that prvalue (and not a temporary) is directly copy-initialized. It de-facto mandates what was known as Return Value Optimization (RVO) and allows non-copyable, non-movable objects to be returned in such a way.

Meanwhile, the other cases of copy elision are still optional. For example, sometimes we want to create an object, set it up and return it.

```
widget setup_widget(int x) {  
    widget w;  
    w.set_x(x);  
    return w;  
}
```

`setup_widget` will copy or move `w` out. Compilers often perform Named Return Value Optimization (NRVO) in such cases, but it is not guaranteed. This situation is unacceptable in these cases, among others:

- If the setup process includes taking a pointer to `w` and storing it elsewhere
- If `widget` is non-copyable non-movable, because its memory location is critically important to its functionality, e.g. `std::mutex`
- If `widget` is non-copyable, because it manages resources that are not easy to copy, and non-movable, because it does not have an empty state, e.g. `open_file` and `not_null_ptr<T>`

On practice, the workaround can be either:

- Two-stage initialization, where a local variable is constructed in its destination (e.g. using the default constructor) and is then immediately passed to function(s) by reference in order to complete the setup of the object
- Always storing the object on heap, e.g. by returning `std::unique_ptr<widget>` instead of `widget` from factory functions

Both "solutions" are often viewed as anti-patterns.

A proper solution should allow for the construct-cook-return pattern, even if a copy or move is not affordable.

§ 2. Proposed solution

If copy elision for a returned variable is allowed, and all non-discarded `return` statements in its potential scope return the variable, then copy elision is guaranteed.

(For the purposes of brevity, the explanation above is not rigorous; see [§ 2.2 Proposed wording](#) for a rigorous explanation.)

§ 2.1. Examples

Constructing and returning a `widget`, guaranteed copy elision applies (since C++17):

```
widget setup_widget(int x) {  
    return widget(x);  
}
```

Constructing, "cooking" and returning a `widget`, guaranteed copy elision applies:

```
widget setup_widget(int x) {  
    auto w = widget(x);  
    int y = process(x);  
    w.set_y(y);  
    return w;  
}
```

A more contrived example where guaranteed copy elision applies:

```
widget setup_widget() {  
    while (...) {  
        auto w = widget(1);  
        if (...) return w;  
        if (...) break;  
        if (...) throw ...;  
        if (...) return w;  
    }  
    return widget(2);  
}
```

An example where guaranteed copy elision does not apply:

```

widget setup_widget() {
    auto w = widget(1);
    if (...) {
        return w;    //!
    } else {
        return widget(2);
    }
}

```

The example above can be "fixed" so that guaranteed copy elision does apply:

```

widget setup_widget() {
    if (...) {
        auto w = widget(1);
        return w;
    } else {
        return widget(2);
    }
}

```

In the following example, `return two;` lies in the potential scope of `one`, so guaranteed copy elision does not apply to `one`:

```

widget test() {
    widget one;
    return one;    //!
    widget two;
    return two;
}

```

Constructing, setting up and passing an object as a parameter using an immediately invoked lambda expression (`consume_widget`'s parameter is directly initialized with `x`):

```

void consume_widget(widget);

void test(int x) {
    int y = process(x);
    consume_widget([&] {
        auto w = widget(x);
        w.set_y(y);
        return w;
    }());
}

```

§ 2.2. Proposed wording

The wording in this section is relative to WG21 draft [\[N4842\]](#).

Add a new subclause of [\[stmt.return\]](#), [\[stmt.return.named\]](#):

It is said that a **return** statement *returns* a variable when its operand is a (possibly parenthesized) *id-expression*, for which the name lookup ([\[basic.lookup\]](#)) finds the variable.

A variable with automatic storage duration is called a **named return object** when all of the following conditions are satisfied:

- the variable is not a function parameter and is not introduced by the *exception-declaration* of a handler ([\[except.handle\]](#)),
- the type of the variable is a class type, is not `volatile`-qualified, and is the same (ignoring cv-qualification) as the return type of the enclosing function, and
- all non-discarded ([\[stmt.if\]](#)) **return** statements in its potential scope ([\[basic.scope\]](#)), of which there is at least one, **return** the variable. [*Note*: The enclosing function cannot be a coroutine. — *end note*]

A [named return object](#) denotes the result object of the function call expression. Statements that **return** a named return object perform no copy-initialization ([\[stmt.return\]](#)) and do not cause the destruction of the object ([\[stmt.jump\]](#)). [*Note*: The *declaration-statement* of a named return object initializes the object denoted by it, see [\[stmt.dcl\]](#). On exit from the scope of a named return object, other than by executing a statement that **returns** it, the object denoted by it is destroyed, see [\[stmt.jump\]](#). During stack unwinding, the object denoted by a named return object is destroyed, see [\[except.ctor\]](#). — *end note*]

Note: The relationship "statement [returns](#) a variable" may be useful in other parts of the standard, e.g. in [\[class.copy.elision\]/3](#).

Note: A [named return object](#) is a variable. The definition carefully avoids mentioning the object it names before stating that the object is the result object of the function call expression (the "return object").

Note: The requirements on a [named return object](#) are intended to be the same as for the optional copy elision in return statements ([\[class.copy.elision\]](#)), except for the last restriction mentioning return statements in the potential scope.

Modify [\[stmt.jump\]/2](#):

On exit from a scope (however accomplished), objects with automatic storage duration that have been constructed in that scope ([excluding the case described in \[stmt.return.named\]](#)) are destroyed in the reverse order of their construction. [*Note*: For temporaries, see [\[class.temporary\]](#). — *end note*] Transfer out of a loop, out of a block, or back past an initialized variable with automatic storage duration involves the destruction of objects with automatic storage duration that are in scope at the point transferred from but not at the point transferred to. (See [\[stmt.dcl\]](#) for transfers into blocks). [...]

Modify [\[stmt.return\]/2](#):

[...] A return statement with any other operand shall be used only in a function whose return type is not cv void; the return statement initializes the glvalue result or prvalue result object of the (explicit or implicit) function call by copy-initialization from the operand ([excluding the case described in \[stmt.return.named\]](#)). [...]

Modify [\[stmt.dcl\]/2](#):

Variables with automatic storage duration are initialized each time their *declaration-statement* is executed. [*Note*: Variables with automatic storage duration declared in the block are destroyed on exit from the block [as described in \[stmt.jump\]](#). — *end note*]

Note: The modified sentence currently duplicates the specification in [\[stmt.jump\]/2](#). If the sentence is turned into a reference, it will not have to duplicate the exception for named return objects.

Modify [\[class.copy.elision\]/1](#):

Copy elision is not permitted where an expression is evaluated in a context requiring a constant expression (`[expr.const]`) and in constant initialization (`[basic.start.static]`). [*Note*: Copy elision might be performed if the same expression is evaluated in another context. `[stmt.return.named]` requires in all contexts what would otherwise be copy elision. — *end note*]

Note: As with "Guaranteed RVO" of [P0135R0](#), "Guaranteed NRVO" is not specified as a special case of copy elision. Nevertheless, the proposed changes will affect the code constructs currently eligible for copy elision. Such copy elision is currently *prohibited* in `constexpr`-related contexts and is *optional* otherwise. With proposed changes, `[stmt.return.named]`, when applies, *requires* copies not to occur, unless the object type is [trivially-copyable](#).

§ 3. Frequently Asked Questions

§ 3.1. Are the proposed changes source or ABI breaking?

The proposal does affect and **can break** constant expressions that rely on effects of the copy-initialization and destruction that are proposed to be elided. The defect report [CWG2278](#), requiring that copy elision is not performed in constant expressions, has been presented in March, 2018.

However, relying on the effects of copy-initialization and destruction in constant expressions is considered exotic, and real-world code breakage is deemed to be minimal.

The proposal is *not* source-breaking outside of constant expressions, because it mandates copy elision in some of the cases that are currently optional.

The proposal is *not* ABI-breaking, because, in all known implementations, whether NRVO is applied for a function does not impact its calling convention.

§ 3.2. What are the costs associated with the proposed changes?

There is no runtime cost associated with the proposed copy elision, because storage for the return object is allocated on stack before the function body starts executing, in all known implementations.

The proposal will make declarations of local variables with automatic storage duration context-dependent: storage of a variable will depend on `return` statements in its potential scope. However, this analysis is local and purely syntactic. The impact on compilation times is thus deemed to be minimal.

Compilers that already do NRVO will enable it (or at least the required part of it) in all compilation modes. The proposal might even have a positive impact on compilation time, because such implementations will not have to check whether copy-initialization on the return type can be performed.

§ 3.3. What about trivially-copyable temporaries?

According to [\[class.temporary\]](#), the implementation is allowed to create a copy when the object of a trivially-copyable type is returned. That is also the case when the copied object participates in "guaranteed RVO" (C++17) or "guaranteed NRVO" (proposed). If the address of such an object is saved to a pointer variable, the pointer will become dangling on return from the function:

```
class A {
public:
    A* p;
    A() : p(this) {}
}

A rvo() {
    return A();
}

A x = rvo();    // a.p may be dangling

A* q{};
A nrvo() {
    A y = A();
    q = &y;
    return y;
}

A z = nrvo();   // z.p may be dangling
                // q may be dangling
```

Changing [\[class.temporary\]](#) and prohibiting such temporaries would cause ABI breakage, and is infeasible.

§ 3.4. Is "named return object" a good term choice?

ISSUE 1 "[Named return object](#)" may not be the best term for our purposes.

It is a mixture of two terms:

- On the one hand, the object subject to NRVO optimization is called a "named return value". However, the term does not belong to the standard. And strictly speaking, a variable names an object, not a value.
- On the other hand, if we allow to shorten "result object of the function call expression" to "returned object" or "return object", then we could say that the variable in question "names the return object". To be precise, it is not an object *named* by something, it is a variable *naming* the return object.

None of those choices is perfect. We could potentially find a better one. Alternatively, the proposed changes could be reworded in a way that does not require the term "named return object".

§ 4. Alternative solutions

§ 4.1. Implement similar functionality using existing features

We can implement similar functionality, with cooperation from the returned object type, in some cases.

Suppose the `widget` class defines the following constructor, among others:

```
template <typename... Args, std::invocable<widget&> Func>
widget(Args&&... args, Func&& func)
: widget(std::forward<Args>(args)...)
{ std::invoke(std::forward<Func>(func)), *this); }
```

We can then use it to observe the result object of a prvalue through a reference before returning it:

```
widget setup_widget(int x) {
    int y = process(x);

    return widget(x, [&](widget& w) {
        w.set_y(y);
    });
}
```

However, it requires cooperation from `widget` and breaks when some of its other constructors accept an invocable parameter. We cannot implement this functionality in general.

§ 4.2. Guarantee NRVO in more cases

```
class builder {
public:
    builder();
    widget build();
    widget rebuild();
    ...
};

widget foo() {
    builder b;
    widget w = b.build();
    if (...) return b.rebuild();
    return w;
}
```

NRVO will **not** be guaranteed for `w`, according to this proposal. Meanwhile, one could say that it could be guaranteed: if the condition is true, then we could arrange it so that `w` (which is stored as the return object) is destroyed before `b.rebuild()` is called.

However, what if `build` saves a pointer to the returned object, which is then used in `rebuild`? Then the `b.rebuild()` call will try to reach for `w`, which will lead to undefined behavior.

While the compiler can in some cases analyze the control flow and usage patterns (usually after inlining is performed), this is impossible in general. (This is why a previous attempt at guaranteeing NRVO was shut down, see [\[CWG2278\]](#).) The limitations of the proposed solution describe the cases where correctness can always be guaranteed without overhead and non-local reasoning.

§ 4.3. Require an explicit mark for named return objects

As an alternative, [named return objects](#) could require a specific attribute or a mark of some sort in order to be eligible for guaranteed copy elision:

```
widget setup_widget(int x) {
    auto w = widget(x) [[nrvo]];
    w.set_x(x);
    return w;
}
```

The benefit of requiring the mark is that the compiler would not have to determine for each local variable whether it could be a named return object. However, the cost of the compile-time checks is deemed to be low, while there would be some language complexity cost associated with the mark.

§ 4.4. Alias expressions

Alias expressions would be a new type of expression. An *alias expression* would accept a prvalue, execute a block, providing that block a "magical" reference to the result object of that prvalue, and the alias expression would itself be a prvalue with the original result object:

```
widget setup_widget(int x) {  
    return using (w = widget()) {  
        w.set_x(x);  
    };  
}
```

Such a construct would require more wording and special cases on the behavior of the "magical" reference `w` and the underlying object. It would be prohibited to `return` from inside the block of the alias expression. More importantly, alias expressions would introduce the precedent of an expression that contains statements, which has issues with a lot of the standard. And as with [explicit marks](#), it introduces more syntax, which the proposed solution avoids.

Alias expressions could also be used to get rid of copies in places other than the return expressions, e.g. when passing a function argument by value:

```
void consume_widget(widget);  
  
void test(int x) {  
    consume_widget(using (w = widget()) {  
        w.set_x(x);  
    });  
}
```

The proposed solution can be used with an immediately invoked lambda expression to perform the same task:

```

void consume_widget(widget);

void test(int x) {
    consume_widget([&] {
        widget w;
        w.set_x(x);
        return w;
    }());
}

```

§ 5. Future work

§ 5.1. Guarantee some other types of copy elision

[\[class.copy.elision\]/1](#) describes 4 cases where copy elision is allowed. Let us review whether it is feasible to guarantee copy elision in those cases:

- (1.1) is feasible to guarantee with the limitations described in this proposal, because such an "optimization" is always correct, does not introduce overhead and does not require non-local reasoning.
- (1.2) leads to an extra allocation in case the control flow escapes the scope before the throw-expression is executed. It would only be feasible to guarantee when the scope contains no other jump statements, and all the functions called are **noexcept**. Those cases are deemed highly specific, but **can** be tackled in a separate proposal.
- (1.3) requires non-local reasoning and is therefore infeasible to guarantee.
- (1.4) covers two cases:
 - When the throw-expression and the try-block reside in different functions. In this case copy elision requires non-local reasoning and is infeasible to guarantee.
 - When they reside in the same function. Such code can be refactored so that this copy elision is unnecessary, unlike (1.1). This use case, if found beneficial enough, **can** be tackled in a separate proposal.

§ 5.2. Guarantee currently disallowed types of copy elision

Requiring copy elision in more cases than is currently allowed by the standard is a breaking change and is out of scope of this proposal. If another proposal that guarantees copy elision in more cases is accepted, those cases could also be reviewed for feasibility of guaranteed copy elision. This proposal will not be influenced by that future work.

§ 5.3. Reduce the number of moves performed in other cases

This proposal belongs to a group of proposals that aim to reduce the number of moves performed in C++ programs. Within that group, there are two subgroups:

- Some proposals allow to replace moves with operations that are yet cheaper than moves (known as relocation or destructive move): [\[N4158\]](#), [\[P0023R0\]](#), [\[P1144R4\]](#).
- Other proposals aim to remove the need for moving altogether. This proposal, as well as [\[P0927R2\]](#), belongs to that group.

The problem solved by the current proposal is orthogonal to the problems dealt with by relocation proposals, as well as to the problem dealt with by P0927R2.

The current proposal combines with [\[P0927R2\]](#) nicely. That proposal requires that the lazy parameter is only used once (and forwarded to another lazy parameter or to its final destination), while in some cases it may be desirable to acquire and use it for some time before forwarding. This proposal would allow to achieve it in a clean way, see the immediately invoked lambda expression example.

The changes proposed by this proposal and [\[P0927R2\]](#), combined, would allow to implement alias expressions (see the corresponding section) without any extra help from the language:

```

template <typename T, invocable<T&> Func>
T also([] -> T value, Func&& func) {
    T computed = value();
    func(computed);
    return computed;
}

void consume_widget(widget);

void test(int x) {
    consume_widget(also(widget(x), [&](auto& w) {
        w.set_x(x);
    }));
}

```

§ 6. Acknowledgements

Thanks to Agustín Bergé, Arthur O’Dwyer, Krystian Stasiowski and everyone else who provided feedback on a draft of this proposal.

§ Index

§ Terms defined by this specification

alias expression, in §4.4

[stmt.return.named], in §2.2

named return object, in §2.2

to return a variable, in §2.2

§ References

§ Normative References

[N4842]

Richard Smith. Working Draft, Standard for Programming Language C++. 27 November 2019.

URL: <https://wg21.link/n4842>

§ Informative References

[CWG2278]

Richard Smith. [Copy elision in constant expressions reconsidered](https://wg21.link/cwg2278). 27 June 2016. drafting. URL: <https://wg21.link/cwg2278>

[N4158]

Pablo Halpern. [Destructive Move \(Rev 1\)](https://wg21.link/n4158). 12 October 2014. URL: <https://wg21.link/n4158>

[P0023R0]

Denis Bider. [Relocator: Efficiently moving objects](https://wg21.link/p0023r0). 8 April 2016. URL: <https://wg21.link/p0023r0>

[P0135R0]

Richard Smith. [Guaranteed copy elision through simplified value categories](https://wg21.link/p0135r0). 27 September 2015. URL: <https://wg21.link/p0135r0>

[P0927R2]

James Dennett, Geoff Romer. [Towards A \(Lazy\) Forwarding Mechanism for C++](https://wg21.link/p0927r2). 5 October 2018. URL: <https://wg21.link/p0927r2>

[P1144R4]

Arthur O'Dwyer. [Object relocation in terms of move plus destroy](https://wg21.link/p1144r4). 17 June 2019. URL: <https://wg21.link/p1144r4>

§ Issues Index

ISSUE 1 ["Named return object"](#) may not be the best term for our purposes. [↩](#)